

TAKE-HOME CODING ASSIGNMENT

AI Engineer

Requirements Gap Analysis Agent

Level	Senior (5+ years experience)
Duration	1–2 days (estimated 6–8 hours of focused work)
Language	Python 3.10+
Submission	GitHub repository (public, or private with reviewer access granted)
Focus	Agentic system design • LLM reasoning • Problem-solving approach

Overview

Engineering teams often talk past each other. Business stakeholders discuss what they need in one set of meetings; engineers discuss how to build it in another. Requirements fall through the cracks, assumptions go unquestioned, and gaps only surface during QA or after launch.

Your task is to build an AI agent that bridges this gap automatically. The agent will:

- **Read a set of business team transcripts** to extract and structure the project requirements being discussed
- **Read a set of engineering team transcripts** to extract the solutions and implementation decisions being planned
- **Analyse both sets** to surface any requirements that are not addressed, under-specified, or potentially misunderstood in the engineering discussions
- **Produce a structured gap report** that a project manager or tech lead could act on immediately

What we're evaluating

We are not looking for a perfect system. We are looking for how you decompose a fuzzy, real-world problem into agents and tools, the reasoning structure you choose, and how honestly you communicate what your system can and cannot do.

Sample Input Data

Your system will be tested against the transcript samples below. You should also create additional transcripts of your own to stress-test edge cases. Transcripts are intentionally informal and messy — just like real meeting notes.

Business Team Transcript — BT-001

Business Team Meeting — Project Orion Kickoff | BT-001

Sarah (PM): Okay, so the core ask from the client is a customer portal. They need their enterprise users to be able to log in, see their account history, and raise support tickets.

James (Client): Right, and the support tickets need to route to the right team automatically. We have three tiers — billing, technical, and general — and each has a different SLA.

Sarah (PM): Got it. What about notifications? Do users need to be kept in the loop on ticket status?

James (Client): Absolutely. Email at minimum. We talked about SMS but that's a nice-to-have. The main thing is they shouldn't have to log back in just to check status.

Sarah (PM): And the account history — how far back?

James (Client): At least 24 months. Ideally filterable by date and type. Oh — and it has to be GDPR compliant. We're operating in the EU.

Sarah (PM): Understood. What about access control? Is this just end users or do admins need a separate view?

James (Client): Admins need to see everything across all accounts, not just their own. And they should be able to impersonate a user for support purposes.

Sarah (PM): Okay. I think that covers the big picture. We'll get back to you with a spec.

Engineering Team Transcript — ET-001

Engineering Team Meeting — Project Orion Sprint Planning | ET-001

Dev (Amir): So for the portal, I'm thinking Next.js on the front end, FastAPI backend. Auth via Auth0.

Dev (Priya): Makes sense. For the ticket system, we could use a simple state machine — open, in-progress, resolved. Routing can be a rules engine based on keywords in the ticket body.

Dev (Amir): Yeah. I was thinking we store tickets in Postgres, account history in the same DB. Should be fine for the scale they're talking about.

Dev (Priya): What about notifications? I can hook up SendGrid for email pretty quickly.

Dev (Amir): Let's do that. We'll skip SMS for now unless they push for it.

Dev (Priya): Agreed. I'll start on the ticket UI. You take auth and account history?

Dev (Amir): Sure. History page will show the last 12 months by default with a load-more for older records.

Dev (Priya): Sounds good. Let's sync Thursday.

 **Note on the sample transcripts**

The samples above contain deliberate gaps. Your agent should identify them. You are also expected to create 3–5 additional transcript pairs of your own to demonstrate that your system generalises beyond a single example.

Tasks

Task 1 — Requirements Extraction Agent

Build an agent (or agentic pipeline) that processes one or more business team transcripts and produces a structured list of requirements. Each requirement should capture:

- A concise requirement statement
- The priority or urgency signalled in the conversation (if any)
- Any constraints or conditions mentioned (SLAs, compliance rules, geographic scope, etc.)
- The speaker or role that raised it (for traceability)

Your agent should handle:

- Multiple speakers with overlapping or contradictory statements
- Requirements mentioned implicitly rather than explicitly
- Distinguishing must-haves from nice-to-haves

Output format is your choice, but it must be machine-readable (JSON is recommended) so that Task 3 can consume it programmatically.

Task 2 — Solution Extraction Agent

Build a complementary agent that processes one or more engineering team transcripts and produces a structured list of planned solutions and implementation decisions. Each solution entry should capture:

- What is being built or decided
- Any explicit scope limitations (e.g. '12 months of history' vs the 24 months requested)
- Technology or tooling choices mentioned
- Anything explicitly deferred or de-scoped

The output of this agent feeds directly into Task 3. Design the schema accordingly.

Task 3 — Gap Analysis Agent

Build a gap analysis agent that takes the structured outputs from Tasks 1 and 2 and produces a gap report. The report should identify:

- **Unaddressed requirements:** requirements from the business transcripts with no corresponding solution in the engineering transcripts
- **Scope mismatches:** cases where engineering has planned something that partially covers a requirement but with different scope or constraints
- **Implicit assumptions:** decisions made in the engineering discussion that may not align with what the business expects, even if not explicitly contradicted
- **Ambiguities:** requirements that are unclear enough that the engineering team could reasonably misinterpret them

Each gap in the report should include:

- A plain-English description of the gap
- The requirement(s) and solution(s) it relates to (by reference)
- A suggested question or action to resolve it



Design note

Think carefully about whether this is best implemented as a single LLM call, a multi-step chain, or a looping agent. There is no single correct answer — but you must justify your choice in the README.

Task 4 — Orchestration & CLI

Wrap your three agents in an orchestration layer that:

- Accepts a directory of transcript files as input (with a flag or config to distinguish business vs engineering transcripts)
- Runs the full pipeline end-to-end and writes the gap report to a file
- Provides a simple CLI so a reviewer can run: `python main.py --business ./transcripts/business --engineering ./transcripts/engineering --output report.json`
- Gracefully handles malformed or empty transcripts without crashing

Optionally, add a `--format` flag that supports both json and markdown output for the gap report.

Task 5 — README & Design Rationale

Your README is a first-class deliverable. It should cover:

- **Architecture overview:** how the three agents relate to each other and how data flows between them
- **Agent design decisions:** why you chose the prompting strategy, model(s), and tool structure you did
- **Known limitations:** what your system does not handle well and why
- **What you would do next:** if you had another day, what would you improve first?
- **Setup & run instructions:** dependency installation, environment variables, and how to run the pipeline

Evaluation Rubric

Criterion	Points	What we look for
Gap analysis quality	30	Does the agent find the real gaps? Are false positives rare? Does it handle scope mismatches and implicit assumptions, not just missing items?
Agent & system design	25	Is the decomposition into agents/tools sensible? Are responsibilities clearly separated? Is the design justified?
Prompt engineering	20	Are prompts clear, structured, and unlikely to produce hallucinated requirements? Is output parsing robust?
Code quality & CLI	15	Readability, error handling, modularity, ease of running
README & design rationale	10	Depth of self-critique, honesty about limitations, clarity of architecture explanation
Bonus: confidence scoring	5	Each gap flagged with a confidence level and reasoning for why it was flagged
Total	100	

Expected Gap Report Output

To illustrate the expected output format, here is an example of a single gap entry your agent should be capable of producing for the sample transcripts above:

```
GAP - Example output entry
gap_id:          "GAP-003"
type:            "scope_mismatch"
```

```
requirement_ref: "REQ-004"  (account history – 24 months, filterable)
solution_ref:    "SOL-003"  (history page – last 12 months with load-more)
description:     "Business requires at least 24 months of account history.
                  Engineering has defaulted to 12 months with a load-more
                  pattern. Filtering by date and type is not mentioned."
suggested_action: "Confirm data retention policy and whether 12-month
                  default meets the client SLA. Add filtering to scope."
```

Constraints & Guidelines

- You may use any LLM provider (OpenAI, Anthropic, Mistral, local via Ollama, etc.). Document your choice and why
- You may use LangChain, LlamaIndex, or any other framework — but be prepared to explain what the framework does for you
- All API keys must be loaded from environment variables. Never hard-code credentials
- The full pipeline must run end-to-end with a single command after following your setup instructions
- Prefer a system that works reliably on the sample transcripts over one that is architecturally ambitious but brittle

Submission Instructions

1. Push your code to a GitHub repository (public or private with reviewer access)
2. Ensure your README is complete and your CLI runs without modification
3. Include at least 3 additional transcript pairs of your own creation alongside the samples provided
4. Include the gap reports your system produces for all transcript pairs
5. Email the repository link to your recruiting contact with subject line: [Senior AI Engineer] Gap Analysis Agent — [Your Name]

Questions

If anything in this brief is unclear, please reach out to your recruiting contact before starting. Ambiguity in the assignment brief is intentional in places — but we would rather you ask than make an assumption that sends you in the wrong direction.

Good luck. We look forward to seeing how you think.